# QWeave: Correctness-First Deterministic and Learned Qubit Routing

**Atharva Sheersh Pandey[1], Diptesh Das[1], Shrivardhini N[1]**
**Haridasu Sreedhar[1], Prof. Bhuvaneswari M[2]**
atharva.sheersh2024@vitstudent.ac.in;
diptesh.das2024@vitstudent.ac.in;
shrivardhini.n2024@vitstudent.ac.in;
haridasu.sreedhar2024@vitstudent.ac.in
[1]School of Computer Science and Engineering, VIT Vellore
[2]Faculty Guide, School of Computer Science and Engineering, VIT Vellore

## Abstract

Restricted two-qubit connectivity forces a compiler to place logical qubits on physical sites and insert movement before nonlocal interactions. This paper studies QWeave as a correctness-first compiler rather than treating low depth alone as sufficient evidence. The deterministic path combines weighted initial mapping, strict local improvement, and shortest-path SWAP routing; identity mapping, Qiskit SABRE, and a small exact initial-mapping oracle are distinct references. We prove termination of the finite mapping descent, preservation of the routing layout invariant and supported unitary semantics, feasibility and fixed-DAG optimality of the list scheduler, equivalence of directed-CNOT lowering, and termination of the guarded learned route. These guarantees do not prove policy quality. That question is evaluated on frozen benchmark v2: 180 synthetic cases, five final training seeds, and 792 held-out outputs. Every tested output passed its applicable numerical semantic check and every direction-lowered cost circuit passed ordered-coupler legality. The paired GNN-PPO minus weighted depth difference was 0 gates with a 95% bootstrap interval of [0,0]; GNN-PPO and no-message-passing PPO also had paired median depth and SWAP differences of 0. Training improved markedly on an untrained policy, but the study supplies no evidence of a message-passing or learned-routing advantage over the deterministic references.

## 1 Introduction

Quantum algorithms are commonly written with unrestricted two-qubit interactions, whereas a device exposes a finite coupling graph. Compilation must therefore choose an initial placement and transform every nonlocal interaction into operations supported by that graph. Qiskit describes layout, routing, translation, optimization, and scheduling as separate stages (IBM Quantum, 2026b). SABRE uses bidirectional heuristic search for layout and SWAP selection (Li et al., 2019); it is consequently a substantive baseline.

QWeave asks a narrower question: under a declared circuit language, topology, mapping convention, and validation protocol, what do deterministic placement, guarded learned routing, and post-routing scheduling each contribute? The answer requires more than an edge-legality check. An emitted circuit can use only valid couplers and still implement the wrong logical computation if a layout update, source SWAP, parameter, or classical destination is mishandled. For that reason, QWeave combines trace replay with numerical checks and keeps mapping, routing, scheduling, and direction lowering as separately reported stages.

This paper contributes (i) an explicit deterministic compiler and adapter contract, (ii) formal correctness arguments for the invariants that can be proved from that contract, (iii) a guarded GNN-PPO implementation with message-passing and training controls, and (iv) a frozen held-out comparison with paired uncertainty estimates and counterexamples. Learning-based routing already exists (Pozzi et

al., 2020; Sinha et al., 2022); neither the use of PPO nor a graph encoder is claimed as novel. The result is also deliberately bounded: the tested policy does not outperform the weighted deterministic reference.

## 2 Related Work

Earlier qubit-allocation systems model placement and movement under architecture constraints. Siraichi et al. formulate qubit allocation as a compiler problem, while Zulehner et al. combine architecture-aware mapping with heuristic search for IBM devices (Siraichi et al., 2018; Zulehner et al., 2019). Cowtan et al. give an architecture-agnostic routing methodology, and Childs et al. study circuit transformations through routing via matchings and token swapping (Cowtan et al., 2019; Childs et al., 2019). These works motivate separating initial placement from dynamic movement.

SABRE combines reverse traversal for initial layout with heuristic SWAP search (Li et al., 2019). In Qiskit, its layout pass may also invoke routing and add idle physical qubits (IBM Quantum, 2026a). Pozzi et al. optimize routing with deep reinforcement learning, while Sinha et al. combine a graph neural network with Monte Carlo tree search (Pozzi et al., 2020; Sinha et al., 2022). QWeave instead uses a masked graph actor-critic trained by PPO, drawing on graph convolution, clipped policy optimization, and generalized advantage estimation (Kipf and Welling, 2017; Schulman et al., 2017; Schulman et al., 2015).

Exact optimization provides a different reference. Nannicini et al. formulate joint assignment and routing by integer programming (Nannicini et al., 2022). QWeave's CP-SAT component is intentionally smaller: it optimizes only a static initial-mapping surrogate for at most eight logical qubits. Its optimal status cannot certify minimum SWAP count or compiled depth.

## 3 Problem Formulation and Deterministic Path

Let C=(g_1,...,g_T) be an ordered supported circuit on logical qubits L=0,...,n-1 and let H=(P,E_H) be a connected undirected routing graph. An initial mapping m_0:L-> P is injective and uses the convention m(logical)=physical. The interaction graph G_C includes isolated logical qubits. An edge (i,j) has weight w_ij equal to its two-qubit interaction count. QWeave minimizes the placement surrogate

$$J(m) = \sum_{(i,j) \text{ in } E_C} w_{ij}\, d_H(m(i),m(j)). \quad (1)$$

where d_H is shortest-path distance. Optional temporal decay replaces an interaction count by ^t, but benchmark v2 uses integer counts.

The mapper places a high weighted-degree logical qubit on a high-centrality physical node, then greedily assigns the remaining logical qubits using interaction weight and partial distance cost. Deterministic ordering resolves ties. Pairwise local search evaluates exchanges of occupied physical sites and accepts the best exchange only when it strictly decreases J.

For each source operation, the deterministic router emits an adjacent one- or two-qubit gate immediately. If a two-qubit gate is nonlocal, it selects a deterministic shortest path and moves the first logical qubit toward the second through edge-local SWAPs. Both forward and inverse layouts are updated after every inserted SWAP. The Basic reference supplies identity mapping to this router; weighted mapping supplies the greedy local-search layout. SABRE is a separate Qiskit compiler path with a fixed seed.

The Qiskit adapter accepts unconditioned one- and two-qubit gates, one-qubit resets, and a terminal measurement suffix while preserving global phase and classical destinations. It rejects barriers, higher-arity gates, dynamic control flow, and classically conditioned operations instead of silently passing unsupported semantics. A trace records one event for each source gate or inserted SWAP. Replay checks source order, parameters, targets, layout updates, and the final mapping. SABRE receives both orientations of every undirected routing edge so all methods share the same adjacency interpretation.

## 4 Formal Correctness Arguments

The propositions below cover algorithmic invariants under the stated input contract. They do not establish that the static objective predicts minimum route cost, that a learned policy is good, or that finite numerical probes are a proof over all inputs.

**Proposition 1 (mapping-search termination).** The strict pair-exchange local search terminates after finitely many accepted moves.

**Proof.**

There are |P|!/(|P|-|L|)! injective maps from L to P, so the search space is finite. Every accepted move produces a previously unvisited objective level with strictly smaller J. A strictly descending sequence over a finite set cannot be infinite. The implementation's iteration cap is therefore an additional guard, not the basis of termination.

**Proposition 2 (layout and legality invariant).** After every emitted operation, the forward layout remains injective, its inverse agrees on occupied sites, and every emitted two-qubit operation uses an edge of E_H.

**Proof.**

The claim holds initially by the mapping contract. An inserted SWAP exchanges the logical occupants of two adjacent physical sites; updating both maps by the same transposition preserves injectivity and inverse agreement. A source one-qubit gate cannot violate edge legality. A source two-qubit gate is emitted only when its current images are adjacent, either immediately or after the shortest-path swaps. Induction over the emitted sequence proves the claim.

**Theorem 1 (supported unitary semantics).** For an equal-width circuit containing only supported unitary one- and two-qubit gates, let R_k be the physical route after the first k source gates, let C_k be the corresponding logical prefix, and let m_k be the current layout. With P_m denoting the wire permutation induced by m,

$$U_{Rk} P_{m0} = P_{mk} U_{Ck}, \text{ for } 0 \le k \le T. \quad (2)$$

**Proof.**

At k=0 both prefixes are identities, so the equality holds. Assume it holds through source gate k. Every inserted physical SWAP permutes two adjacent wires, and the layout update records exactly that same transposition. The physical state interpreted through the updated layout therefore represents the same logical state. Once the operands are adjacent, applying the source gate to their current physical images has the same action as applying the logical gate before the current wire permutation. Composing these steps with the induction hypothesis proves the invariant for k+1. At k=T, the final routed unitary equals the logical unitary up to the declared initial and final permutations.

**Proposition 3 (fixed-route scheduling).** For positive operation durations and no resource constraints beyond physical qubit exclusivity, the list scheduler returns a feasible schedule. Its makespan equals the weighted critical-path length of the fixed per-qubit precedence DAG and is therefore minimal among schedules that preserve that DAG.

**Proof.**

The DAG contains an edge between successive operations on each physical qubit. Consequently, incomparable ready operations share no qubit and may overlap. For operation i, the scheduler uses

$$r_i = d_i + \max_{j \text{ in succ}(i)} r_j;$$
$$s_i = \max_{j \text{ in pred}(i)} (s_j + d_j). \quad (3)$$

with an empty maximum equal to zero. Thus each operation starts after every predecessor finishes, proving feasibility. Every schedule respecting the DAG has makespan at least its weighted critical path. Earliest-start recursion attains that bound, so the returned makespan is optimal for this fixed DAG. This statement does not allow gate commutation or model control electronics.

**Proposition 4 (directed lowering).** If at least one orientation of an undirected routing edge supports CNOT, the lowering pass implements either requested CNOT orientation and an edge-local SWAP using only declared directed CNOTs and one-qubit Hadamards.

**Proof.**

For a supported physical CNOT directed from v to u, the opposite orientation satisfies

$$CX_{u\to v} = (H_u \times H_v)\, CX_{v\to u}\, (H_u \times H_v). \quad (4)$$

An undirected SWAP is three alternating CNOTs. Replacing each unavailable orientation by Equation (4) preserves the product unitary and leaves every two-qubit instruction on an allowed ordered arc.

**Proposition 5 (guarded-route termination).** For a finite supported circuit on a finite connected routing graph, the guarded learned environment terminates and returns a legal route.

**Proof.**
Each policy-controlled segment has a finite step limit. On repeated nonprogress or exhaustion, fallback selects a finite shortest path, performs adjacent swaps, and executes the current gate. Hence fallback advances the source index. A finite circuit permits only finitely many such advances, and Proposition 2 gives legality for every fallback operation. Replay validation must succeed before the environment returns a result.

## 5 Learned Router

The Gymnasium environment exposes an Execute action and one SWAP action for every sorted hardware edge. Its observation contains five features per physical node, four per hardware edge, a five-value frontier description, the current logical-to-physical mapping, and a binary legal-action mask. Illegal actions do not mutate the circuit or layout. Fallback counts are reported because the termination guard can conceal a weak policy.

The actor-critic performs two rounds of edge-conditioned message passing and scores every edge SWAP plus Execute. PPO uses clipped policy ratios, generalized advantage estimation, value and entropy terms, and gradient clipping (Schulman et al., 2017; Schulman et al., 2015). A zero-message-passing PPO control preserves the optimizer and reward, while an untrained graph control preserves the architecture without PPO updates. All variants start from the weighted deterministic map, making this a routing-policy comparison rather than a learned-layout experiment.

## 6 Validation and Experimental Protocol

Structural replay checks mapping completeness, injectivity, source-gate order, inserted SWAP consistency, global phase, and edge legality. For equal-width unitary circuits of at most six qubits, QWeave evaluates Equation (2) at k=T as an exact operator equality. Wider unitary circuits and outputs with idle physical sites use reproducible layout-aware statevector probes up to 12 physical qubits. Exact equality is a semantic proof for the evaluated circuit; a finite probe set is evidence that can detect faults but is not a universal proof. Reset and terminal-measurement circuits receive structural replay. Branch-dependent dynamic control remains unsupported.

After routing, the fixed-route scheduler measures depth or declared-duration makespan on the same legal route. A separate hardware-cost pass lowers CNOTs and SWAPs to declared directed couplers, validates ordered arcs, then evaluates the schedule and an independent-error proxy. Durations and edge errors are synthetic assumptions rather than device calibrations.

Benchmark version 2.0 freezes 180 cases at four, six, and eight logical qubits, with two instances per family. Sixty training cases use GHZ, layered-CX, seeded-interaction, Bernstein-Vazirani, and QAOA MaxCut families; the latter follows the alternating-operator construction of Farhi et al. (2014). Eighty-four validation cases test topology transfer and held-out families. The 36 untouched test cases combine QFT circuits, motivated by the approximate transform of Coppersmith (2002), and hardware-efficient circuits with two-row grids, chorded rings, and larger grids containing idle sites. Every case stores QASM, source hash, undirected and directed edges, and declared synthetic error values.

Three candidates receive 1,024 training steps and validation seed 101. The selection score is median depth plus half the SWAP count plus five times the fallback count. The selected candidate is trained for 4,096 steps at seeds 7, 17, 29, 41, and 53 for both GNN-PPO and the no-message control. The untrained control uses the same initialization seeds. SABRE uses the same five seeds; Basic and weighted routing are

deterministic. Runtime is the median of three measurements after one warmup. Seeds are collapsed within each case before paired differences are formed. Confidence intervals use 5,000 deterministic bootstrap resamples with seed 20260921.

## 7 Results

A fresh local run passed 69 tests under Python 3.11.15 and Qiskit 2.5.2. The aggressive validation candidate scored 19.5; conservative and standard each scored 24.0. Table 1 summarizes the 36 held-out cases. For seeded methods, each case contributes its within-case seed median. All 792 outputs passed their applicable numerical check. QWeave routes also passed symbolic replay; SABRE has no QWeave trace. Every unit-duration scheduling delta was zero, as expected when circuit depth already expresses the same per-qubit precedence.

| Method | Depth | SWAP | Runtime | Fallback |
|---|---|---|---|---|
| Basic | 48.5 | 11.5 | 0.0022 | 0 |
| Weighted | 36.0 | 0.0 | 0.0036 | 0 |
| SABRE | 46.0 | 7.0 | 0.0028 | 0 |
| GNN-PPO | 37.5 | 0.0 | 0.3473 | 4543 |
| No-message PPO | 36.0 | 0.0 | 0.1040 | 742 |
| Untrained GNN | 198.0 | 193.0 | 0.6681 | 8059 |

Table 1 Benchmark-v2 test summary. Runtime is median wall time in seconds.

The paired GNN-PPO minus weighted depth difference was 0 gates with interval [0,0]; versus SABRE it was 0 with interval [0,5]. Its paired SWAP difference versus weighted routing was 0 with interval [0,5]. GNN-PPO and no-message PPO had paired median depth and SWAP differences of 0 with intervals [0,0], so the experiment supplies no evidence that message passing helped. Training did matter relative to the untrained GNN: the paired depth difference was -162.5 with interval [-184,-106], and the SWAP difference was -175 with interval [-223,-148].

Family results explain the aggregate. On hardware-efficient circuits, weighted, GNN-PPO, and no-message PPO all had median depth 27 and no SWAPs. On QFT, weighted routing had median depth 71.5 and 16.5 SWAPs, GNN-PPO had 83.5 and 41, and no-message PPO had 90 and 31.5. GNN-PPO improved by eight depth layers on qft_i1_6q_chorded_ring, but regressed by 89 depth gates and 130 SWAPs on qft_i1_8q_grid2. It invoked 4,543 test fallbacks, compared with 742 for no-message PPO and zero for deterministic methods. Seed 29 was unstable, ending with 212 training fallbacks and a recent mean return near -103.

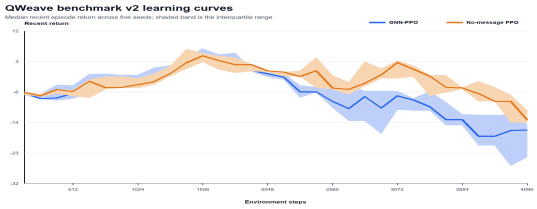



Figure 1 Five-seed learning curves. Lines show the median recent episode return and bands show the interquartile range.

Before direction lowering, most routes were illegal on the one-way couplers. After reverse-CNOT and SWAP synthesis, all 792 cost circuits passed ordered-arc legality. Median added depth was 32 for weighted routing, 34 for SABRE, 37.5 for GNN-PPO, 28.5 for no-message PPO, 57 for Basic, and 441 for the untrained GNN. The corrected paired GNN-PPO minus weighted makespan difference was 0 ns with interval [0,1113]. These synthetic proxy costs provide no learned advantage.

## 8 Threats to Validity

The formal results depend on the supported gate contract, a connected routing graph, correct primitive gate implementations, and the absence of unmodelled shared hardware resources. They prove preservation properties of the algorithms, not software free of defects; tests and independent reruns address implementation risk. Numerical probes for wider circuits are incomplete by construction. Measurement, reset, and dynamic-control semantics do not receive the same exact unitary argument.

The empirical study uses synthetic circuits, durations, and error rates. Its five seeds and 36 held-out cases support paired statements about benchmark v2, not general performance or state-of-the-art rank. Runtime is machine-specific, and fallback makes the learned methods safe while also weakening the interpretation of policy-only behavior. The retained pre-correction record is disclosed because its duration fields scheduled pre-lowering circuits; the schema-2.1 replay recomputed only direction-dependent cost fields from saved checkpoints and frozen cases.

## 9 Conclusion

QWeave combines an explicit deterministic reference, a small mapping oracle, fixed-route scheduling, directed lowering, and a guarded learned router. Formal arguments now state why strict mapping descent terminates, why routing preserves the declared layout semantics, why the fixed-DAG scheduler is feasible and makespan-optimal under its model, why direction reversal is unitarily sound, and why fallback terminates. Benchmark v2 complements those proofs with held-out evidence. PPO training clearly improves on an untrained policy, but the tested message-passing component and learned route do not improve on weighted routing. Larger preregistered device-derived studies are needed before any superiority or calibrated-fidelity claim.

## References


Childs, A. M., Schoute, E., and Unsal, C. M. (2019). Circuit Transformations for Quantum Architectures. TQC 2019, LIPIcs 135, 3:1-3:24. doi:10.4230/LIPIcs.TQC.2019.3.

Coppersmith, D. (2002). An Approximate Fourier Transform Useful in Quantum Factoring. arXiv:quant-ph/0201067.

Cowtan, A., Dilkes, S., Duncan, R., Krajenbrink, A., Simmons, W., and Sivarajah, S. (2019). On the Qubit Routing Problem. TQC 2019, LIPIcs 135, 5:1-5:32. doi:10.4230/LIPIcs.TQC.2019.5.

Farhi, E., Goldstone, J., and Gutmann, S. (2014). A Quantum Approximate Optimization Algorithm. arXiv:1411.4028.

IBM Quantum. (2026a). SabreLayout API Reference. IBM Quantum Documentation.

IBM Quantum. (2026b). Transpiler Stages. IBM Quantum Documentation.

Kipf, T. N., and Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. ICLR 2017. arXiv:1609.02907.

Li, G., Ding, Y., and Xie, Y. (2019). Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices. ASPLOS 2019. arXiv:1809.02573.

Nannicini, G., Bishop, L. S., Gunluk, O., and Jurcevic, P. (2022). Optimal Qubit Assignment and Routing via Integer Programming. ACM Transactions on Quantum Computing, 4(1). doi:10.1145/3544563.

Pozzi, M. G., Herbert, S. J., Sengupta, A., and Mullins, R. D. (2020). Using Reinforcement Learning to Perform Qubit Routing in Quantum Compilers. arXiv:2007.15957.

Schulman, J., Moritz, P., Levine, S., Jordan, M. I., and Abbeel, P. (2015). High-Dimensional Continuous Control Using Generalized Advantage Estimation. arXiv:1506.02438.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.

Sinha, A., Azad, U., and Singh, H. (2022). Qubit Routing Using Graph Neural Network Aided Monte Carlo Tree Search. AAAI 2022. arXiv:2104.01992.

Siraichi, M. Y., dos Santos, V. F., Collange, S., and Pereira, F. M. Q. (2018). Qubit Allocation. CGO 2018, 113-125. doi:10.1145/3168822.

Zulehner, A., Paler, A., and Wille, R. (2019). An Efficient Methodology for Mapping Quantum Circuits to the IBM QX Architectures. IEEE TCAD, 38(7), 1226-1236. doi:10.1109/TCAD.2018.2846658.